



AIDA — Feature reference

Every screen, what it does, and who can do what

AI & Data Academy · 30 September 2026

Generated from docs/FEATURES.md, which stays the source of truth.

Every screen in the platform, in the order it appears in the sidebar, with what it does and who can do what on it.

Two conventions used throughout:

- **Route** is the page in the app (`/org`), **API** is the endpoint behind it (`GET /api/analytics/hr`). Every API path is prefixed with `/api`.
 - **Who** lists the roles. Where a rule is narrower than a role — "the author", "the trainer who owns it", "the named approver" — that is stated instead, because ownership beats rank on most objects here.
-

0. Roles, scopes and how access is decided

The ladder

All authorisation predicates live in one place, `backend/app/core/rbac.py`, so a rule is defined once rather than reimplemented per module.

user < trainer < manager < bu_head < hr < hr_lead < admin

Role	What it is	Reach
user	A collaborator	Themselves
trainer	Runs trainings	The trainings they own
manager	Runs a team (N+1)	Their team
bu_head	Runs a business unit	Every team in their BU
hr	HRBP	The BUs explicitly assigned to them
hr_lead	L&D / HR lead	The whole organisation, plus money
admin	Platform administrator	Everything, including the platform itself

A machine caller can present the `X-Admin-Token` header instead of a role; it is treated as a platform admin. It exists for scripts and scheduled jobs.

The named permission groups

Predicate	Roles	Used for
can_curate	trainer, manager, bu_head, hr, hr_lead, admin	Creating shared content: skills, role profiles, pathways, trainings; assigning a course
OVERSEER_ROLES	manager, bu_head, hr, hr_lead, admin	Following people rather than content
can_read_reporting	hr, hr_lead, admin	The analytics and compliance surfaces
can_see_cost	hr_lead, admin	Any money figure
can_onboard	hr_lead, admin	Creating a collaborator's account
is_people_admin	hr, hr_lead, admin	Acting on any person or team
is_platform_admin	admin	Labs, roles, integrations, deletions

Two separations are deliberate and worth knowing before reading the rest:

L&D is not a platform administrator. `hr_lead` runs people and reporting; they do not get labs, role grants or deletions. Conflating the two is how "HR can see everything" quietly becomes "HR can change anything".

Money is narrower than people. Cost is `hr_lead` and `admin` only. A manager deciding a training request sees the training, not the price — the budget line is L&D's call, and showing an approver the number invites them to decide on the number instead of on the person.

The two scopes

`reporting_scope` answers "which people may this viewer see in a report":

- `admin`, `hr_lead` — the whole organisation.
- `hr` — the BUs assigned to them. **An HRBP with no assignment sees nobody**, not everybody; granting the widest access by omission is backwards for a payload carrying matricules.
- Everyone else — nobody.

`oversight_scope` is wider and is used for follow-up views rather than HR reporting: it falls back to `reporting_scope`, then gives a `bu_head` their BUs, a manager or team lead their teams, and everyone else only themselves.

Navigation is not security

`frontend/src/lib/nav.ts` filters the sidebar by role, but the API refuses these routes on its own and must keep doing so — a hidden link is still a reachable URL. What the filter fixes is the other failure: a collaborator reading "Administration" in their sidebar, clicking it, and being told no.

1. Overview

1.1 Dashboard

Route `/` · Who everyone

The learning home. It composes five things:

- **Onboarding wizard** — shown once, on first connection, to a learner whose `onboarded` flag is still false. It asks a few questions and proposes a starting path. Its content is editable by L&D (§1.9).
- **Quests** — the short list of what to do next.
- **Recommendations** — AI-suggested content for this learner (`GET /api/ai/recommend`).
- **For You** — a personalised feed of trainings, sessions and challenges.
- **Leaderboard snippet** and role-filtered shortcuts into the rest of the app.

1.2 Schedule

Route `/schedule` · Who everyone

Every upcoming training session and event, with registration.

Action	API	Who
See what is coming	<code>GET /api/events/upcoming</code> , <code>GET /api/training-sessions</code>	Everyone
Register / cancel	<code>POST /api/training-sessions/{id}/register</code> · <code>/cancel</code>	The learner themselves
Add to calendar	<code>GET /api/training-sessions/{id}/ics</code>	Everyone
Schedule a session	<code>POST /api/training-sessions</code>	<code>can_curate</code>
Edit / delete a session	<code>PUT /DELETE /api/training-sessions/{id}</code>	The session's trainer, or admin
See the roster	<code>GET /api/training-sessions/{id}/roster</code>	The session's trainer, or admin
Take attendance	<code>POST /api/training-sessions/{id}/attendance</code>	The session's trainer, or admin

Registration closes when the session says it does, and a session attached to a training can be restricted to that training's enrolled learners.

Attendance matters beyond the register: a confirmed présence contributes the session's **scheduled duration** to the learning-hours total, which is the most defensible kind of time in the whole system — measured, not estimated.

1.3 Approvals

Route `/approvals` · Who everyone (the page shows a different half to each side)

Both sides of every decision on one screen. An approver gets a queue of what is waiting on them plus the history of what they already decided, declines and reasons included. A collaborator gets the same records from their side: what they asked for and where it stands.

Three things go through approval: a training request, a shared asset, and a declared learning record.

The chain for a training request is two stages, created up front so the requester sees the whole path rather than discovering a second approver after the first says yes:

1. **The N+1** — the manager of the requester's team.
2. **The BU head** — who owns the budget line the manager is spending.

Someone who both heads the BU and manages the team decides once, as they would in the room. A stage whose approver cannot be resolved is created with no approver and **stays pending**: a BU with no head means the request is

stuck and somebody needs to appoint one. Skipping the stage would silently approve a spend nobody signed off.

Action	API	Who
Raise a request	POST /api/approvals/requests	Any learner
Withdraw one	DELETE /api/approvals/requests/{id}	The requester only
See my requests	GET /api/approvals/mine	The requester
See my queue	GET /api/approvals/inbox	The named approver
Decide	POST /api/approvals/{kind}/{id}/decide	The named approver for the current stage; hr , hr_lead and admin may act on any stage to unblock a chain

A manager cannot reach past their stage into the BU head's, and a BU head cannot decide before the manager has.

Training requests are generic. The price is withheld from anyone outside `can_see_cost`, in the request list, in the picker used to raise one, and in the notification the approver receives. The figure is still stored on the request for L&D's spend reporting — it is simply not served to people who may not see money.

1.4 My Team

Route /team · **Who** manager, bu_head, hr, hr_lead, admin

The team dashboard: members, their progress, XP, activity and assigned work.

Action	API	Who
Open a team dashboard	GET /api/teams/{id}/dashboard	The team's lead or manager, HR, admin
Add / remove members	POST / DELETE /api/teams/{id}/members/...	The team's lead, or admin
Assign a training to the team	POST /api/teams/{id}/assign-formation	The team's lead or manager, or admin
Create / edit / delete a team	POST / PUT / DELETE /api/teams/{id}	HR or admin
Org-wide team overview	GET /api/teams/overview	HR or admin

1.5 Organization — HR analytics

Route /org · **Who** hr, hr_lead, admin

The reporting board, scoped server-side: an HRBP's view contains only their BUs, and the page says so with a badge rather than letting a partial view be read as the whole company.

The source switch

One control at the top decides which platform's numbers the whole screen reports:

Source	What it shows
AIDA	In-platform learning only — lessons, attended sessions, declared learning
Coursera	The provider's own population, including the people with no account here
Both	The two together, each hour counted once

This exists because the blended total is genuinely ambiguous: `learning_hours` already contained Coursera hours, so "AIDA" and "Coursera" could not simply be added. The backend now reports `app_hours` and `external_hours` separately and each mode quotes the right one. Man-days, the hours chart and the exports all follow the switch.

The Coursera source deliberately reports the provider's whole population rather than the matched share — reporting only the linked accounts would understate it by an order of magnitude. The gap is a headline figure, not a footnote:

"254 of 261 Coursera accounts are not linked to an AIDA account, so their hours stay out of the HR board."

The tabs

Tab	What it holds	Who
Teams	The reporting line as an org chart, BU heads and HRBPs included	hr, hr_lead, admin
Content analytics	One row per training and course: subscribers, completion, attendance, hours, rating, feedback rate, skills, and cost	Cost column: hr_lead , admin only
Per collaborator	One row per person, filtered by BU / practice / location / job level, with Coursera columns in Both mode and a link to their provider profile	hr, hr_lead, admin
Skill coverage	The skill heatmap across the org	hr, hr_lead, admin
HRBP perimeters	Which BUs each HRBP covers, editable	hr_lead , admin only

Exports

GET </api/analytics/hr/export?fmt=xlsx|pdf&source=app|coursera|both>

The file follows the switch, and its name carries the source ([aida-rapport-both-2026-09-30.xlsx](#)).

- **AIDA** — Synthèse, Programmes, Collaborateurs. Hours are in-platform only.
- **Both** — the same, plus eight Coursera columns per person and a summary that names each platform separately.
- **Coursera** — a different workbook: Synthèse, Par programme, Par BU, Par partenaire, Top contenus, Personnes, with four charts.

A column hidden on screen is absent from the workbook too — cost is removed, not zeroed, because a 0 MAD training budget is a claim and a wrong one.

One scoping caveat: the Coursera export is not BU-scoped, exactly like the Coursera panel on screen, because the provider does not tell us which of our BUs most of those accounts belong to. The AIDA and Both exports stay scoped.

One person's Coursera profile

Route </coursera/{email}> · Who hr, hr_lead, admin

Opened from the link in the collaborator table. It carries both halves of the person: an **AIDA** block (completed, in progress, hours, XP with the Coursera-credited share, certificates) above the **Coursera** one (enrolments, completion, hours, grades per course, certificates, programme breakdown, points to note). Two actions live here:

- **Download the learning card** as a branded PDF.
- **Link the provider account** to an AIDA one. Linking backfills the certificates already collected, fills blank profile fields from the provider (never overwriting something L&D set), and grants XP — 10 XP/hour, 40 per completion, 25 per certificate. It is idempotent: re-linking replaces its own contribution instead of stacking another.

1.6 Reports — the report builder

Route </reports> · Who hr, hr_lead, admin

Pick a dataset, drop columns into *group by* and *measure*, filter, choose a chart, save it under a name. The layout follows that order left to right, because it is the order the question gets asked in.

Action	API	Who
List / preview datasets	GET /api/reports/datasets/preview	HR, L&D, admin (rows scoped by reporting_scope)
Upload a dataset	POST /api/reports/datasets	HR, L&D, admin
Delete a dataset	DELETE /api/reports/datasets/{id}	HR, L&D, admin — AIDA's own built-in datasets cannot be deleted

Action	API	Who
Run a query	POST /api/reports/datasets/{id}/query	HR, L&D, admin
Save / list views	POST / GET /api/reports/views	HR, L&D, admin
Delete a view	DELETE /api/reports/views/{id}	The view's owner only
Export a query	POST /api/reports/datasets/{id}/export	xlsx or pdf; cost columns follow <code>can_see_cost</code>

1.7 Mandatory

Route /mandatory · **Who** hr, hr_lead, admin · **API** GET /api/compliance/mandatory

Every obligatory course and training **that was actually assigned to someone**, and where each person got to: not started, in progress, done, overdue.

Only assigned people are counted. Someone who took a mandatory course nobody gave them is a volunteer, and counting volunteers as compliance is how a report comes out green while the obligation is unmet.

Rows are filtered by `oversight_scope`, so a BU head sees their BU and L&D sees the organisation.

1.8 Governance console

Route /admin/governance · **Who** hr_lead, admin

The organisation as something you edit, in two tabs.

Organisation — the structure: business units, who heads them, which HRBPs cover them, and the teams inside. It is edited in place on a chart that looks like the read-only one, so nobody has to re-learn where things are.

Action	API
Create / rename / reorder / archive a BU	POST PATCH DELETE /api/governance/bus/..., /bus/reorder
Appoint a BU head	PUT /api/governance/bus/{id}/head
Assign HRBPs to a BU	PUT /api/governance/bus/{id}/hrbps
Create / edit / delete a team	POST PATCH DELETE /api/governance/teams/...

People — the roster: role, title, BU, practice, location, matricule, job level, and the invitation flow for a new colleague.

Action	API	Who
List and edit people	GET / PATCH /api/governance/people/...	hr_lead, admin
Invite a collaborator	POST /api/governance/invitations (preview at /invitations/preview)	hr_lead, admin
Deactivate someone	POST /api/governance/people/{id}/deactivate	hr_lead, admin
Grant the admin role	same endpoint	Platform admin only — L&D can set every other role but cannot mint an administrator

1.9 Onboarding console

Route /admin/onboarding · **Who** hr_lead, admin · **API** GET/PUT /api/onboarding/config

Edits the very first screen a new colleague sees: the roles they can pick, the goals, the wording.

Nothing here is ever deleted — a retired choice is hidden instead, because past learners still carry its key on their profile and in analytics. Deleting it would rewrite their history rather than stop offering the option.

2. Learn

2.1 Trainings

Route `/formations` · **Who** everyone reads

Instructor-led trainings with a curriculum of lessons, live sessions, an attendance register, AI playgrounds, graded challenges and a certificate.

Action	API	Who
Browse and open	<code>GET /api/formations/{id}</code>	Everyone
Create	<code>POST /api/formations</code>	<code>can_curate</code>
Edit / delete	<code>PUT / DELETE /api/formations/{id}</code>	The training's trainer, or admin
Invite trainees	<code>POST /api/formations/{id}/invite</code>	The trainer, or admin
Accept / decline an invitation	<code>POST /api/formations/{id}/respond</code>	The invited learner
Enrol	<code>POST /api/formations/{id}/enroll</code>	Any learner — unless the training is invite-only, which needs an invitation or a join code
Complete a lesson	<code>POST /api/formations/{id}/lessons/{lid}/complete</code>	Enrolled learners only
Run a prompt playground	<code>.../playground</code>	Enrolled learners; needs AI configured
Attempt a graded challenge	<code>.../challenge</code>	Enrolled learners; LLM-judged
See assessment results	<code>GET /api/formations/{id}/assessments</code>	The trainer, HR, or admin
See the roster	<code>GET /api/formations/{id}/roster</code>	The trainer, or admin
Schedule / cancel a session	<code>POST / DELETE /api/formations/{id}/sessions/...</code>	The trainer, or admin
Download the certificate	<code>GET /api/formations/{id}/certificate</code>	The learner who completed it

Lesson durations are authored, so a completed lesson contributes its real length to the learning-hours total rather than a flat constant.

2.2 Pathways

Route `/pathways` · **Who** everyone reads

Curated journeys that string together trainings, courses and certifications.

Action	API	Who
Browse	<code>GET /api/pathways</code>	Everyone
Create	<code>POST /api/pathways</code>	<code>can_curate</code>
Edit / delete	<code>PUT / DELETE /api/pathways/{id}</code>	The creator, HR, or admin
Enrol	<code>POST /api/pathways/{id}/enroll</code>	Any learner
Assign to a team	<code>POST /api/pathways/{id}/assign</code>	The team's lead or manager, HR, or admin

2.3 Skills

Route `/skills` · **Who** everyone reads

A skill graph with follow, self-rating, peer and manager ratings, role profiles and gap analysis.

Action	API	Who
Browse skills and linked content	GET <code>/api/skills/{id}/content</code>	Everyone
Follow a skill, self-rate	POST <code>/api/skills/{id}/follow, /rate</code>	Any learner, on their own skills
Create a skill	POST <code>/api/skills</code>	<code>can_curate</code>
Link content to a skill	POST <code>/api/skills/{id}/link</code>	<code>can_curate</code>
Give a peer rating	POST <code>/api/skills/{id}/rate</code>	Any learner, on someone else
Give a manager rating	same	That person's manager or lead, HR, or admin
Record an assessment result	same	HR or admin
Define a role profile	POST <code>/api/skills/profiles</code>	<code>can_curate</code>
Team gap matrix	GET <code>/api/skills/gap</code>	The team's lead or manager, HR, or admin
Org heatmap	GET <code>/api/skills/heatmap</code>	HR or admin

Nobody can post a manager rating on themselves, and self-ratings go through the self-rating path so the two cannot be confused.

2.4 Courses

Route `/courses` · **Who** everyone reads

Self-paced courses, in-app or external (Coursera and other providers).

Action	API	Who
Browse, open, follow lessons	GET <code>/api/courses/{id}</code> , POST <code>.../complete</code>	Everyone
Create	POST <code>/api/courses</code>	Any signed-in learner (see the note below)
Edit / delete	PUT / DELETE <code>/api/courses/{id}</code>	The author, or admin
Assign to a person	POST <code>/api/courses/{id}/assign</code>	<code>can_curate</code>
Track an assignment	GET <code>/api/courses/{id}/tracking</code>	<code>can_curate</code>

Assigning a course sends the person a notification and a branded email, records whether it is mandatory and when it is due, and makes them count in the Mandatory report (§1.7) and on their own record (§4.2).

A locked course returns **423** rather than a silent refusal, so the caller is told *why* rather than just *no*.

Note — an asymmetry worth knowing: creating a course, a challenge, a shared asset or a session is open to any signed-in learner, while *assigning* a course is restricted to curators. That is defensible (posting your own material is participation; putting work on somebody else's plate is not) but it does mean the catalogue is not gated. If the catalogue should be curated, the create endpoints are where to add `can_curate`.

2.5 Labs

Route `/labs` · **Who** everyone

Hands-on, graded lessons run against a live sandbox, with difficulty tiers, XP per step and badges. Signing in saves progress and earns XP; browsing does not require it.

Lab definitions themselves are platform administration — see §5.1.

2.6 Stacks

Route `/stacks` · **Who** everyone

Read-only recipes for running tools locally. `GET /api/stacks, /{id}`.

3. Collaborate

3.1 Challenges

Route [/challenges](#) · **Who** everyone

An open innovation board: a brief, submissions against it, and votes.

Action	API	Who
Browse, filter by tag or search	GET /api/challenges	Everyone
Post a challenge	POST /api/challenges	Any learner
Edit a challenge	PUT /api/challenges/{id}	The author, or admin
Open / close it	POST /api/challenges/{id}/status	The owner, or admin
Delete it	DELETE /api/challenges/{id}	The owner, or admin
Submit	POST /api/challenges/{id}/submissions	Any learner
Vote	POST /api/submissions/{id}/vote	Any learner
Delete a submission	DELETE /api/submissions/{id}	Its author, or admin

Editing in place exists because deleting and re-posting also threw away the submissions and votes attached to the brief — so in practice a wrong date stayed wrong.

3.2 Sessions — sharing and decks

Route [/sessions](#) · **Who** everyone

Internal talks with their slide decks, searchable and filterable by tag. A PowerPoint deck uploaded here is converted so it can be read in the browser as a PDF.

Action	API	Who
Browse / search / filter	GET /api/sharing	Everyone
Post a session	POST /api/sharing	Any learner
Upload a deck	POST /api/sharing/upload	Any learner — max 60 MB
Read or download the deck	GET /api/sharing/{id}/deck	Everyone
Edit, deck included	PUT /api/sharing/{id}	The author, or admin
Delete	DELETE /api/sharing/{id}	The author, or admin

Replacing a deck is an edit like any other — slides get fixed after the talk — and the alternative was deleting the session and losing its URL.

3.3 Assets

Route [/assets](#) · **Who** everyone

Shared notebooks, models and artefacts, with a review step.

Action	API	Who
Browse / search / filter	GET /api/assets	Everyone
Publish an asset	POST /api/assets	Any learner
Edit	PUT /api/assets/{id}	The author, or admin

Action	API	Who
Delete	DELETE /api/assets/{id}	The author, or admin
Review (approve / reject)	POST /api/assets/{id}/review	The author's team lead or manager, HR, or admin

An approved asset that is then edited materially returns to **pending**, because an approval has to mean the text that is actually on display.

3.4 Certifications

Route /certifications · **Who** everyone reads

Three things on one page: the catalogue, suggestions to team members, and certificates people have earned.

Action	API	Who
Browse the catalogue	GET /api/certifications	Everyone
Add an entry to the catalogue	POST /api/certifications	can_curate
Remove an entry	DELETE /api/certifications/{id}	Admin only
Suggest a certification to someone	POST /api/certifications/{id}/suggest	Overseers, within their oversight_scope
Withdraw a suggestion	DELETE /api/certifications/suggestions/{id}	The suggester, or admin
Share an earned certificate	POST /api/certifications/earned (file via /upload)	Any learner, for themselves
Read the certificate file	GET /api/certifications/earned/{id}/file	Everyone
Delete an earned certificate	DELETE /api/certifications/earned/{id}	Its owner, or admin
Expiring soon	GET /api/certifications/expiring	Filtered by oversight_scope
Compliance view	GET /api/certifications/compliance	can_read_reporting, filtered by oversight_scope

Expiry is derived from the certification's validity period, so a certificate recorded without an end date still ages correctly. Certificates collected from Coursera are recorded on the learner's AIDA profile when their account is linked — which is why the same number can legitimately appear in both the AIDA and the Coursera certificate columns.

4. Me

4.1 Settings

Route `/settings` · **Who** everyone, for themselves

Account details, interface language (fr / en), and sign-in state — which identity provider is configured, which session is active, and sign-out.

The language choice is stored on the profile, not only in the browser, so the emails and notifications the backend generates arrive in the language the person chose in the UI.

4.2 My learning

Route `/history` · **Who** everyone, for themselves · **API GET** `/api/history?learner_id=`

One person's whole record, from every source at once: trainings, courses, labs, Coursera, and anything they declared by hand — plus their certificates.

Before this, the record was scattered across five tables and each screen showed its own slice, so nobody — including the learner — could answer "what have I actually done, and what is still open".

Every item arrives in the same shape, so the list sorts and filters across sources: filter by status (completed / in progress / not started) and by kind (training, course, lab, external, declared). Mandatory items still open, and anything overdue, are called out at the top. Items with no date sort last rather than pretending to be old.

Declared learning is entered here too (**POST** `/api/learning/records`) and can be verified by that person's manager or lead, HR, or an admin — never by themselves.

4.3 My Profile

Route `/profile` · **Who** everyone, for themselves

Level ring and XP, daily streak, badges and achievement tiers, the skill tree, open quests, and earned certificates with expiry badges.

4.4 Leaderboard

Route `/leaderboard` · **Who** everyone · **API GET** `/api/leaderboard`

XP and badge rankings across the platform.

5. Tools

5.1 Admin

Route `/admin` · **Who** admin only

Platform administration proper — narrower than governance, because running the people is not running the app. Platform stats, plus three consoles:

Labs — `/admin/labs`

Author, validate, edit and delete lab definitions; reload them from files. `GET/POST/PUT/DELETE /api/admin/labs/...`

Learners — `/admin/learners`

List accounts, set a role, edit an HR profile, delete an account, and add a recruit. Role changes and deletions are admin-only; `POST /api/admin/recruits` additionally accepts `hr_lead` through `can_onboard`.

Coursera — `/admin/coursera`

Two independent halves, because they need different things:

Half	What it does	Credentials
Public catalogue	Preview a course by its slug and import it with its real cover art and workload	None needed
Enterprise	Connection status, import enrolled learners, link an account, run a full sync	Enterprise API credentials

`GET /api/admin/coursera/status, /lookup, POST /import, /link, /import-enrolled, /sync`.

Also here: `POST /api/admin/digest/run` and `POST /api/admin/reminders/certifications`, which fire the scheduled jobs on demand.

5.2 Not in the sidebar

Two surfaces exist but are not advertised in navigation:

- `/connections` — database connections for the data-quality tooling: create, test, list tables, profile a table. `GET/POST/DELETE /api/connections/...`
- `/studio` — the data-quality workbench (suites, checks, runs: `/api/suites`, `/api/checks`, `/api/runs`). Its nav entry is commented out in `nav.ts`.

6. Cross-cutting

6.1 Notifications and email

An in-app bell ([GET /api/notifications](#), [POST /api/notifications/read](#)) plus branded HTML email through Mailpit in development. Notifications are raised on invitations, course assignments, approval steps, session reminders, certificate expiry and the weekly digest. They are written in the recipient's own language, using [app/core/i18n.py](#).

6.2 AI

[app/ai/providers.py](#) supports Anthropic, Azure OpenAI and a local Ollama, and resolves them in that order depending on what is configured. Features that use it:

Feature	Endpoint
Tutor chat, with streaming	POST /api/ai/tutor,/tutor/stream
Content recommendations	GET /api/ai/recommend
Quiz generation	POST /api/ai/quiz
Code / answer review	POST /api/ai/review
Check suggestions	POST /api/ai/suggest-checks
Result explanations	POST /api/ai/results/{id}/explain
Written Coursera analysis	POST /api/analytics/coursera/report

Where AI is unavailable the written analysis falls back to a computed narrative and says which one you are reading, rather than failing or quietly inventing one.

6.3 Periodic reports

[GET /api/analytics/coursera/report.pdf](#) and [/person.pdf](#) produce branded PDFs for a month, a quarter or a year — teal palette, logo, charts, findings and AI commentary. [ref](#) is any date inside the period wanted, so "the quarter that just closed" is a date rather than an off-by-one argument about quarter numbering.

The committee report deliberately carries **no named list of people to chase**. It reports the aggregate instead, and the named detail goes to the one manager who can act on it, as a per-person card.

6.4 The social layer

Comments, likes, shares and star ratings attach to any content type through one set of endpoints ([/api/social/{type}/{id}/...](#)). Anyone may comment, like, share and rate; a comment can be deleted by its author, HR, or an admin.

6.5 Background jobs

Celery beat runs three:

Job	What it does
coursera_sync	Pulls the enterprise enrolment report nightly
certification_reminders	Warns people whose certificates are about to expire
weekly_digest	Sends the weekly summary email

Each can also be fired by hand from the admin console.

6.6 Identity and sign-in

Microsoft Azure SSO is the primary path when configured; a handle-based login stays available as a demo fallback, and [/login](#) says which one is active. [IdentityMiddleware](#) rewrites a [learner_id](#) in the query string from the

token whenever authentication is enabled, so a caller cannot ask for somebody else's data by editing the URL.

Current environment: SSO is switched off and the demo accounts are enabled, which also disables that middleware. Re-enabling authentication is what turns the identity guarantee back on.

6.7 Languages

The whole interface is bilingual (French and English). `t(key)` never renders a raw key: a missing translation falls back to English, then to the literal the caller passed, so a partially translated screen degrades to readable text rather than to `nav.dashboard`.

Appendix A — Role capability matrix

Yes = full · Part = limited or scoped · — = none

Capability	user	trainer	manager	bu_head	hr	hr_lead	admin
Learn, enrol, complete, earn XP	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Post a challenge / asset / session / course	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Raise a training request	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Create trainings, pathways, skills, role profiles	—	Yes	Yes	Yes	Yes	Yes	Yes
Assign a course to a person	—	Yes	Yes	Yes	Yes	Yes	Yes
Run a training (roster, sessions, assessments)	—	Part own	—	—	Part	—	Yes
Decide an approval	—	—	Part stage 1	Part stage 2	Yes any	Yes any	Yes any
Team dashboard, assign work to a team	—	—	Part own	Part own BU	Yes	Yes	Yes
Review a shared asset	—	—	Part own team	Part own BU	Yes	Yes	Yes
Suggest a certification	—	—	Part scope	Part scope	Yes	Yes	Yes
Verify a declared learning record	—	—	Part own team	Part own BU	Yes	Yes	Yes
HR analytics, Coursera analytics, exports	—	—	—	—	Part own BUs	Yes	Yes
Mandatory / compliance reporting	—	—	—	—	Part own BUs	Yes	Yes
Report builder	—	—	—	—	Part own BUs	Yes	Yes
See cost and budget	—	—	—	—	—	Yes	Yes
Create / invite / deactivate a collaborator	—	—	—	—	—	Yes	Yes
Governance: BUs, heads, HRBPs, teams, titles	—	—	—	—	—	Yes	Yes
Edit the onboarding wizard	—	—	—	—	—	Yes	Yes
Create / edit / delete teams	—	—	—	—	Yes	Yes	Yes
Grant the admin role	—	—	—	—	—	—	Yes
Labs authoring, integrations, account deletion	—	—	—	—	—	—	Yes

Appendix B — Demo accounts

Sign in with the handle at `/login` while SSO is off.

Handle	Role
<code>admin, abdelghafour lahrache</code>	admin
<code>aicha.abouaid</code>	hr_lead (L&D)
<code>khalid.ou, maryam.bakh</code>	hr (HRBP)
<code>mostapha.aibi</code>	bu_head — AI & Data
<code>omar.elouafi, leila.senhaji, hicham.raji</code>	manager
<code>abdelghafour.lahrache, salma.idrissi, karim.mansouri, rachid.berrada, fatima.benjelloun, amina</code>	trainer
<code>salma.elbarbori</code> , and the rest of the roster	user

To see the role boundaries working, the quickest tour is: raise a training request as `salma.elbarbori`, watch it appear without a price in `omar.elouafi`'s queue, then open the same request as `aicha.abouaid` and see the cost.